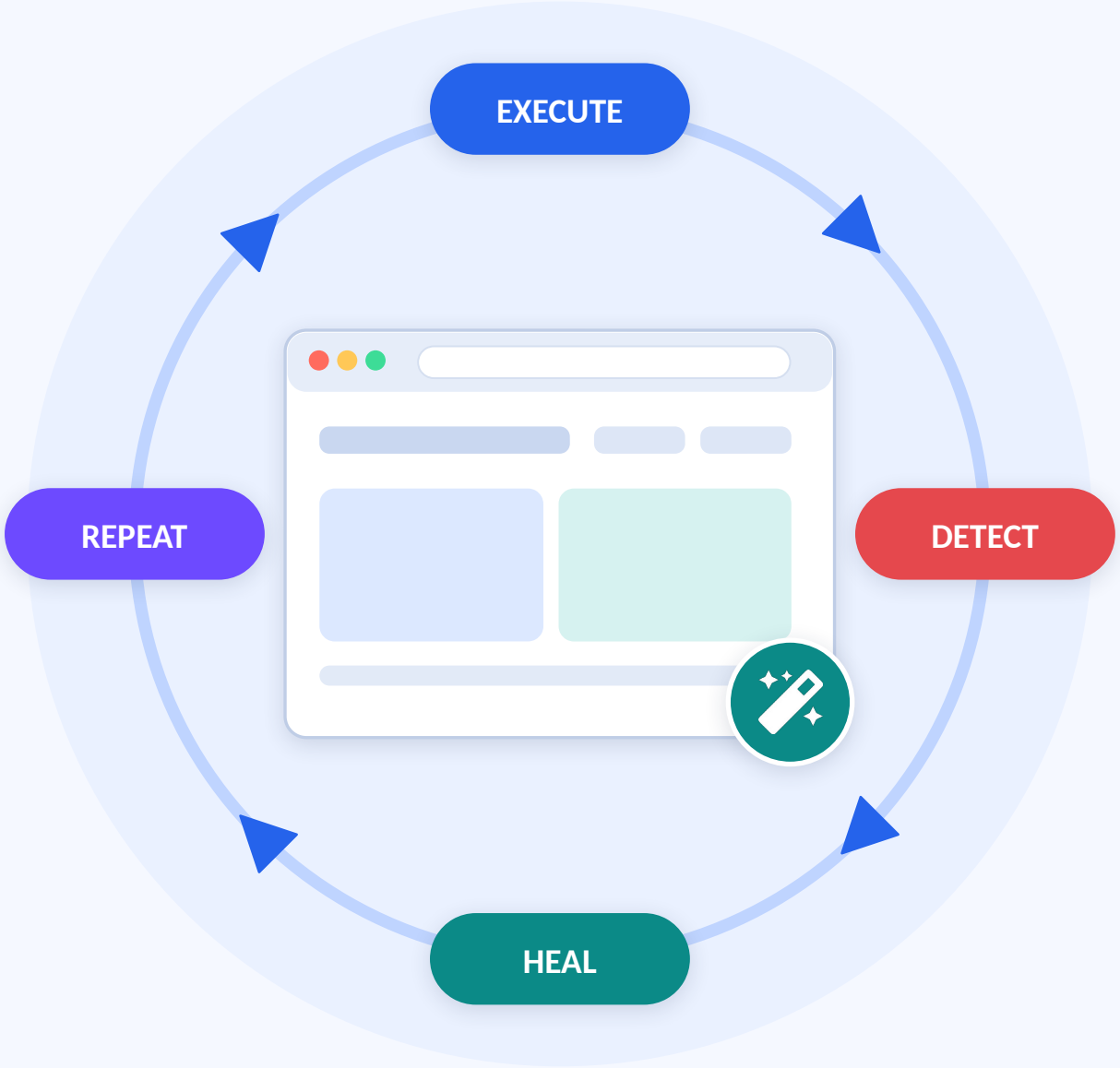
AutoPatch

Self-Healing Browser Automation

Automate Once. Adapt When Things Change.

An AI-powered automation system that detects broken browser workflows, repairs them, verifies the fix, and reuses the repaired workflow automatically.

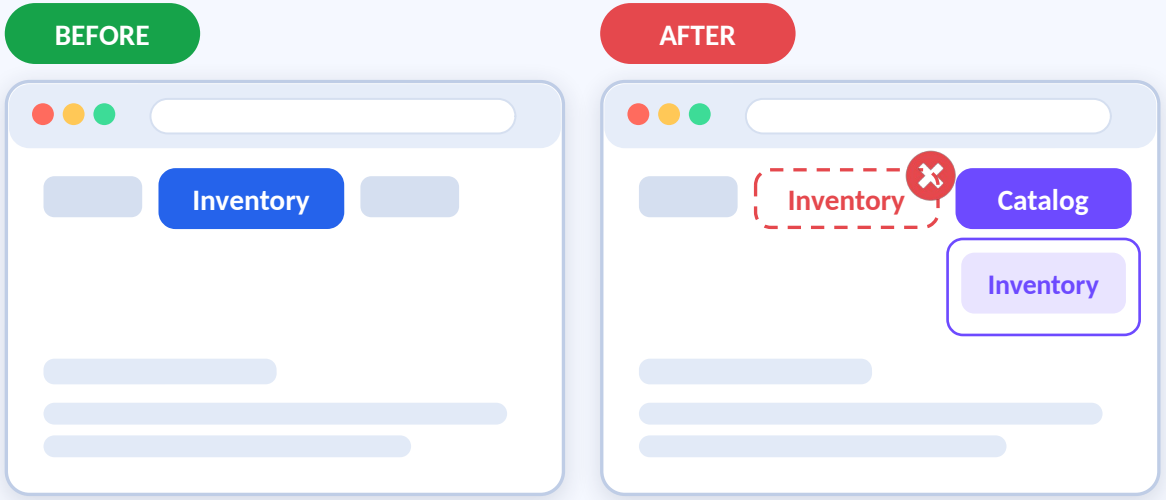


Kritical Hacks 2026 | AI/ML Track | Powered by TinyFish + Playwright

Websites Evolve. Automations Break.

Browser scripts follow specific selectors and navigation paths; websites do not promise to keep them unchanged.

- Renamed / relocated buttons
- Changed DOM structure or locators
- New pop-ups and intermediate steps
- Redesigned page flows



UI CHANGE → FAILED SCRIPT → MANUAL DEBUGGING → WORKFLOW DOWNTIME

How do we retain the speed of scripted automation while making workflows adapt to interface changes on their own?

[1]: Playwright locators and trace documentation explain how selectors locate elements and how failed runs can be diagnosed.

Today, Teams Trade Speed for Adaptability.

TRADITIONAL PLAYWRIGHT	Fast, repeatable runs	Breaks when a relied-upon UI path changes
AI BROWSER AGENTS	Flexible navigation and reasoning	Repeated agent execution for recurring tasks
AUTOPATCH	Playwright for repeat runs + TinyFish for discovery and recovery	Verified repairs become the next default workflow

The missing link is PERSISTENT SELF-HEALING: recover once, verify once, benefit on every future run.

Meet AutoPatch: An Automation That Repairs Itself.

Describe a browser task once. AutoPatch builds a reusable workflow, executes it efficiently, and heals it when the interface changes.

01 DISCOVER

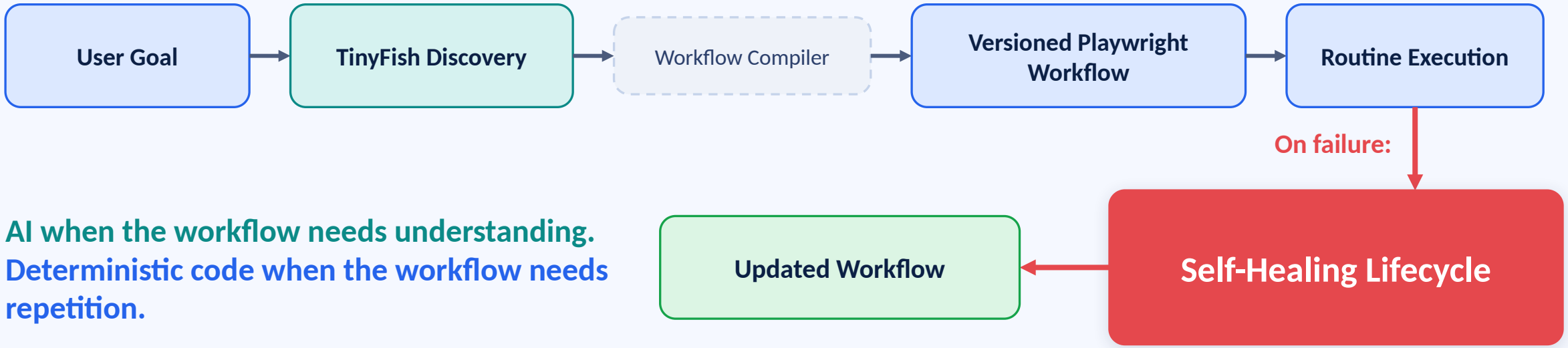
TinyFish understands the goal and explores the site.

02 EXECUTE

Workflow compiler produces reusable Playwright instructions; Playwright handles regular runs.

03 SELF-HEAL

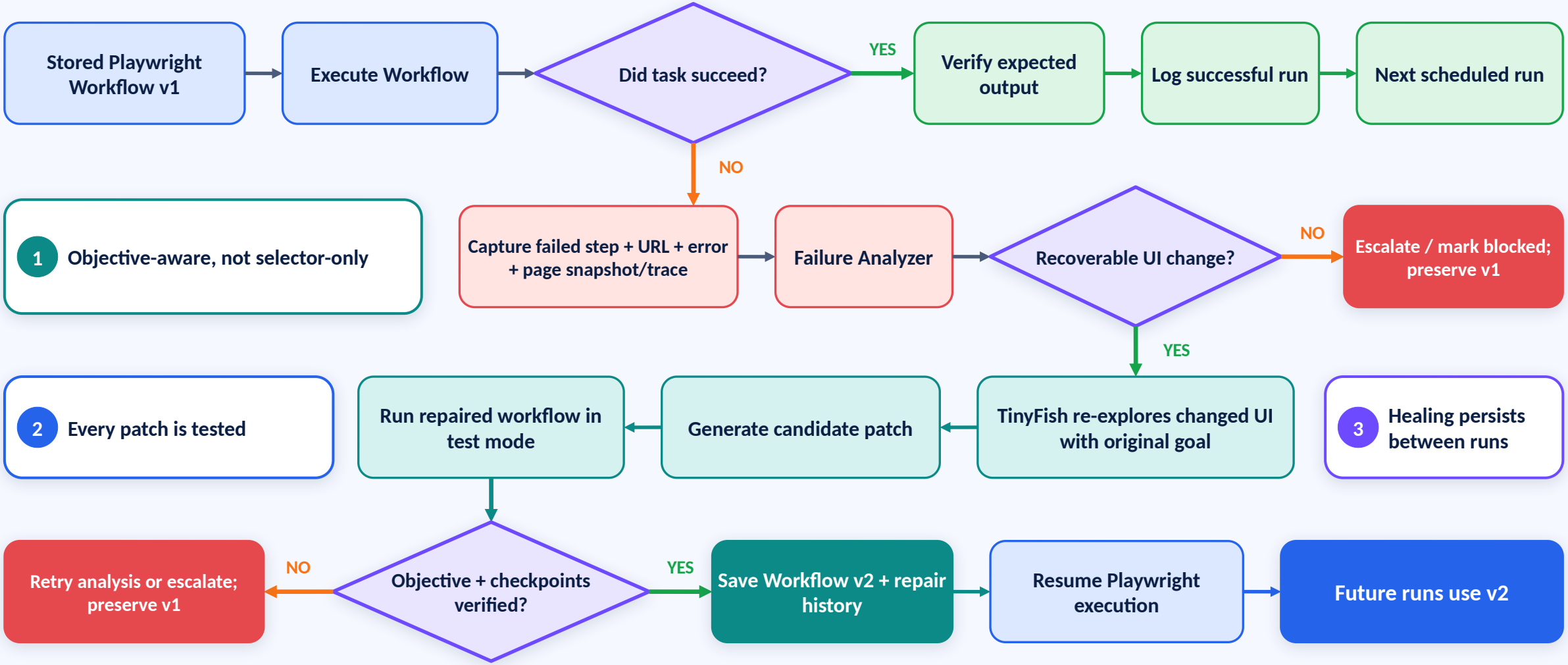
When execution fails, TinyFish re-explores, a patch is generated and tested, and the corrected workflow is saved.



AI when the workflow needs understanding. Deterministic code when the workflow needs repetition.

When a Workflow Breaks, AutoPatch Gets to Work.

Detect → Diagnose → Explore → Patch → Verify → Persist → Resume



A successful repair is not a temporary workaround — it is the new reusable automation.

TinyFish Is AutoPatch's Intelligence Layer.

Intelligent recovery **TinyFish**

WHEN AI IS USED

Discover: Understand a natural-language task and navigate the target site.

Diagnose: Interpret failure context against the original objective.

Recover: Explore the changed interface and identify a new valid path.

Repair: Supply observations for a candidate script patch.

Efficient execution

WHEN AI IS NOT NEEDED

- Execute stored scripts using Playwright.
- Check deterministic assertions and expected outputs.
- Save versioned workflows, run logs, and accepted repairs.

Proposed TinyFish integration — request/response:

Input to TinyFish:
{target_url, original_goal, failed_step, error_context, current_page_context, expected_outcome}

→ **TinyFish agent:** navigate + reason over the changed website.

→ **Output to AutoPatch:**
{recovered_navigation_path, observed_target_elements, task_result, recovery_notes}

AutoPatch: TinyFish observations → Patch Generator → Playwright validation → Versioned workflow

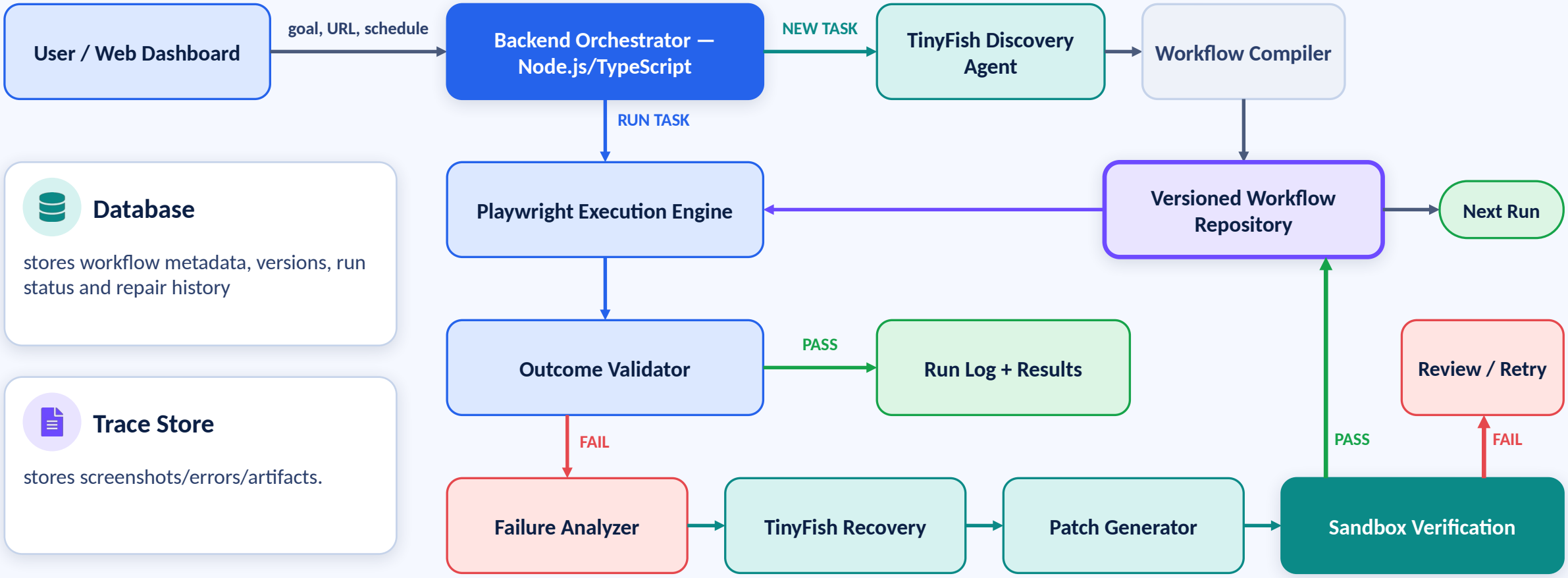
TinyFish specific features: Natural-language web navigation; multi-step browser tasks; structured results/progress events [2].

AI technique: Agentic reasoning with pretrained models, grounded in live webpage observations. No claim of a newly trained custom model.

[2] TinyFish Documentation — https://docs.tinyfish.ai/key-concepts/endpoints

Hybrid Agentic + Deterministic Architecture.

React / Next.js | Node.js + TypeScript | TinyFish API | Playwright | PostgreSQL (or SQLite for MVP) | Object storage for logs/traces



Security note: Credentials remain in a secure secret store; high-impact actions can require approval.

[1] Playwright Locators — https://playwright.dev/docs/locators [3] Playwright Trace Viewer — https://playwright.dev/docs/trace-viewer

Built for Teams Whose Workflows Cannot Keep Breaking.

Featured scenario:

Supplier Inventory Collection

Primary user:

Developers and operations teams maintaining recurring web automations.

Secondary users:

E-commerce operations, data extraction teams, QA engineers, back-office automation owners.

1 CREATE

"Collect item name, SKU and stock status from the supplier portal every morning."

2 RUN

AutoPatch executes saved Playwright workflow against Portal Version A.

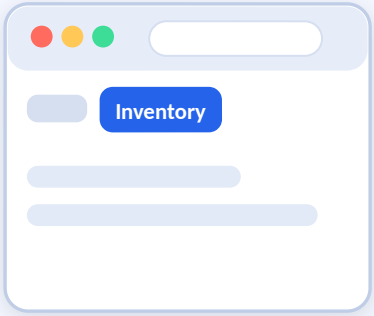
3 BREAK

Supplier redesigns portal: "Inventory" moves under "Catalog"; old navigation fails.

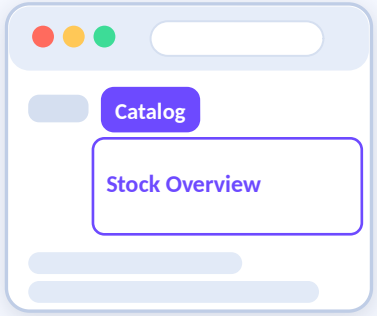
4 HEAL

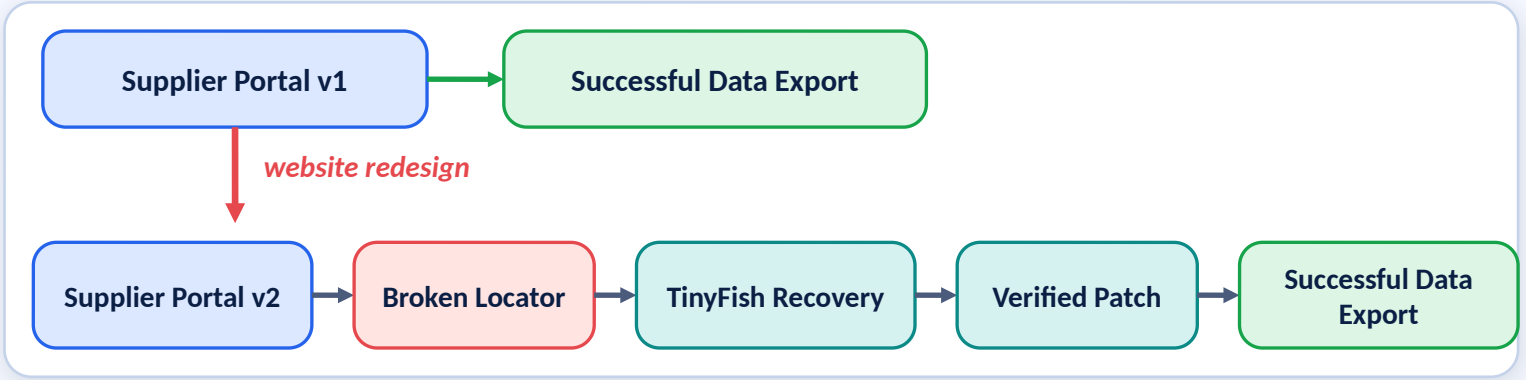
TinyFish finds "Catalog → Stock Overview"; AutoPatch patches, verifies the extracted SKU/stock fields, saves Version B.

BEFORE



AFTER





Tomorrow's run uses Version B directly — no repeat discovery needed.

Less Manual Repair. More Reliable Automation.

Expected impact (outcomes, not fabricated metrics)

- **Reduced hands-on maintenance for supported UI changes.**
- **Faster return to a working workflow after interface changes.**
- **Fewer repeat agent calls during unchanged recurring executions.**
- **Traceable, reversible repair history.**

Measurement scorecard (TARGET METRICS, not achieved results)

- **Repair success rate** = validated auto-repairs / eligible UI-change failures.
- **Mean time to recovery** = time from first failure to approved verified patch.
- **Reuse rate** = post-repair runs completed without re-invoking the agent.
- **Correctness** = successful output checks / repaired workflow executions.

24-HOUR BUILD PLAN:

0–4h	4–9h	9–15h	15–20h	20–24h
Demo site versions + baseline Playwright + success assertions.	Execution monitor + failure capture + version storage.	TinyFish recovery trigger + candidate patch generation.	Verification loop + persist/replay patched workflow.	Dashboard polish + controlled live demonstration + results capture.

Feasibility proof points: Browser automation, tracing and assertions from Playwright [1,3]; web-agent navigation from TinyFish [2]; versioned storage via established backend tools.

[1] Playwright Locators — https://playwright.dev/docs/locators [3] Playwright Trace Viewer — https://playwright.dev/docs/trace-viewer

[2] TinyFish Documentation — https://docs.tinyfish.ai/key-concepts/endpoints

Sources confirm platform capabilities. Architecture, diagrams, use case, schedule and anticipated impact above are the AutoPatch team's proposed design.

From Self-Healing Scripts to Self-Maintaining Automation.

One Workflow Heals Itself → Many Complex Workflows Heal → Team-Wide Automation Reliability → Platform-Scale Self-Maintenance

MVP

Detect, repair, verify, and persist a changed navigation/locator workflow.

NEXT

Multi-page repair, smarter workflow compilation, and repair confidence scoring.

SCALE

Shared repair patterns, CI/CD integrations, enterprise approval policies, multi-site workflows.

END STATE

Autonomous workflow reliability layer for evolving websites.

The future of automation is not just running tasks automatically. It is keeping them running automatically.

